

Operations Research Project

Nour JAZI

May 7, 2026

Repository name: Op_Research_Projet_JAZI_Nour

Course: Operations Research

Abstract

This project implements several classical flow algorithms from Operations Research, and to detect negative cycles in residual graphs. The implementation does not use any flow library such as NetworkX. All algorithms are implemented manually using a residual graph structure. Graphviz is used to visualize the final results.

Contents

1	Introduction	3
2	Project Requirements	3
3	Graph and Residual Graph Structure	3
3.1	Graph Representation	3
3.2	Residual Graph	4
4	Maximum Flow	4
4.1	Ford-Fulkerson / Edmonds-Karp	4
4.2	Maximum Flow Result	5
5	Minimum Cut	6
6	Minimum Cost Flow	6
6.1	Problem Definition	6
6.2	Repeated Shortest Augmenting Paths	7
7	Bellman-Ford Version	7
8	Dijkstra with Reduced Costs	7
9	Minimum Cost Flow Result	8
10	Negative Cycle Detection	9

11 Most Vital Arc	9
12 Commands Used	10
13 Difficulties Encountered	10
14 Conclusion	11

1 Introduction

The goal of this project is to implement and test flow algorithms studied in the Operations Research course. The project focuses on three main problems:

- Maximum flow and minimum cut using Ford-Fulkerson / Edmonds-Karp.
- Minimum cost flow using repeated shortest augmenting paths.
- Detection of negative cycles in the residual graph.

The project also includes Graphviz visualizations in order to make the results easier to understand and verify visually.

Personally, I found that the most important concept in the project is the **residual graph**. Once the residual graph is correctly implemented, the different algorithms become much easier to connect together.

2 Project Requirements

The project required the implementation of:

1. Ford-Fulkerson algorithm for maximum flow and minimum cut.
2. Minimum cost flow with repeated augmenting paths:
 - (a) using Bellman-Ford for negative costs,
 - (b) using Dijkstra with reduced costs and potentials.
3. Negative cycle detection.
4. A report and a README file.
5. Graph visualization using Graphviz.

It was forbidden to use an existing flow library. Therefore, the graph, residual graph, shortest path algorithms and flow algorithms were implemented manually.

3 Graph and Residual Graph Structure

3.1 Graph Representation

The graph is represented using an adjacency list. Each node stores a list of outgoing edges. Each edge contains:

- destination node,
- capacity,
- cost,
- current flow,

- reference to the reverse edge,
- information about whether the edge is original or residual.

The basic edge structure used in the implementation is:

```

1 class Edge:
2     def __init__(self, to, rev, capacity, cost=0):
3         self.to = to
4         self.rev = rev
5         self.capacity = capacity
6         self.cost = cost
7         self.flow = 0

```

3.2 Residual Graph

For each original edge (u, v) , two residual edges are created:

- a forward edge with residual capacity:

$$u(u, v) - f(u, v)$$

- a backward edge with residual capacity:

$$f(u, v)$$

The backward edge is essential because it allows the algorithm to cancel part of a previous decision. For example, if a flow was sent through an edge and later a better path is found, the residual graph allows the algorithm to reduce this previous flow.

For minimum cost flow, residual costs are also used:

- forward residual edge cost: $c(u, v)$,
- backward residual edge cost: $-c(u, v)$.

This structure is used by all algorithms in the project.

4 Maximum Flow

4.1 Ford-Fulkerson / Edmonds-Karp

The maximum flow problem consists of sending as much flow as possible from a source s to a sink t while respecting capacity constraints.

The implemented version uses BFS to find augmenting paths. This corresponds to the Edmonds-Karp version of Ford-Fulkerson.

The algorithm is:

1. Start with zero flow.
2. Search an augmenting path from s to t in the residual graph.

3. Compute the bottleneck capacity of the path.
4. Augment the flow along the path.
5. Repeat until there is no augmenting path.

When no path from s to t exists in the residual graph, the current flow is maximum.

4.2 Maximum Flow Result

The professor's test instance for maximum flow is:

Start node	End node	Capacity
0	1	20
0	2	30
0	3	10
1	2	40
1	4	30
2	3	10
2	4	20
3	2	5
3	4	20

Table 1: Maximum flow test instance

The maximum flow is computed from node 0 to node 4.

Arc	Flow	Capacity
$0 \rightarrow 1$	20	20
$0 \rightarrow 2$	30	30
$0 \rightarrow 3$	10	10
$1 \rightarrow 2$	0	40
$1 \rightarrow 4$	20	30
$2 \rightarrow 3$	10	10
$2 \rightarrow 4$	20	20
$3 \rightarrow 2$	0	5
$3 \rightarrow 4$	20	20

Table 2: Maximum flow result

The maximum flow value is:

60

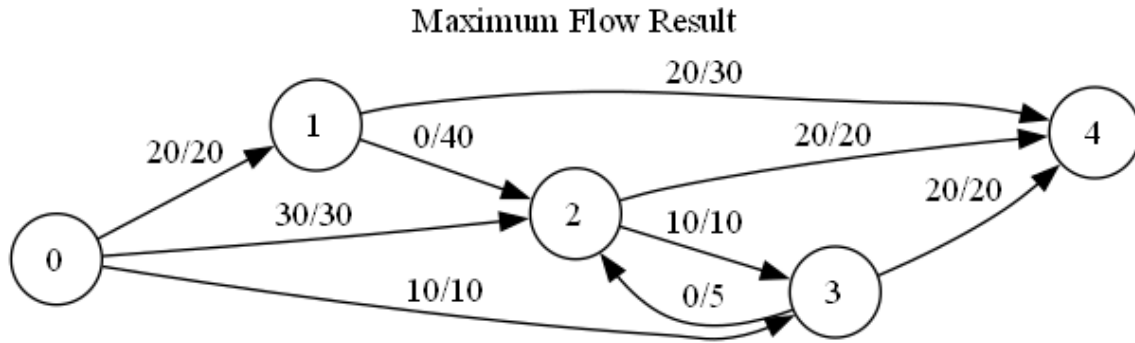


Figure 1: Graphviz visualization of the maximum flow result

Personal observation: this visualization is useful because saturated edges are immediately visible. For example, the edges leaving node 0 are all saturated, which explains why the maximum flow cannot be increased.

5 Minimum Cut

After the maximum flow is found, the minimum cut is obtained by exploring the residual graph from the source.

Let S be the set of nodes reachable from the source in the residual graph. Let T be the remaining nodes. The minimum cut is formed by original edges from S to T .

For the maximum flow example:

$$S = \{0\}$$

$$T = \{1, 2, 3, 4\}$$

The cut edges are:

$$(0, 1), (0, 2), (0, 3)$$

The cut capacity is:

$$20 + 30 + 10 = 60$$

Therefore:

$$\text{Maximum flow} = \text{Minimum cut} = 60$$

This confirms the max-flow min-cut theorem.

6 Minimum Cost Flow

6.1 Problem Definition

In the minimum cost flow problem, each edge has:

- a capacity,

- a cost per unit of flow.

The objective is to send a required amount of flow from source to sink with minimum total cost.

The total cost is:

$$\sum_{(u,v)} f(u,v) \times c(u,v)$$

6.2 Repeated Shortest Augmenting Paths

The implemented method repeatedly sends flow along the cheapest path in the residual graph.

At each iteration:

1. compute a shortest path in the residual graph,
2. send the maximum possible flow along this path,
3. update the residual graph,
4. repeat until the required demand is sent.

7 Bellman-Ford Version

Bellman-Ford is used because the residual graph may contain negative cost edges. This happens because every original edge has a reverse residual edge with opposite cost.

Bellman-Ford can handle negative edges as long as there is no negative cycle.

The complexity of Bellman-Ford is:

$$O(VE)$$

per shortest path computation.

8 Dijkstra with Reduced Costs

Dijkstra cannot be used directly when negative costs exist. To solve this problem, potentials are introduced.

For an edge (u, v) , the reduced cost is:

$$c_R(u, v) = c(u, v) + \pi(u) - \pi(v)$$

where π is the potential function.

The goal is to make all reduced costs non-negative, so Dijkstra can be applied.

After each Dijkstra execution, the potentials are updated:

$$\pi(v) = \pi(v) + d(v)$$

The real final cost is still computed using the original costs, not the reduced costs.

Personal note: this was the most subtle part of the project. The algorithm does not change the real problem; it only transforms edge costs during shortest path computations so that Dijkstra becomes valid.

9 Minimum Cost Flow Result

The professor's minimum cost flow instance is:

Start node	End node	Capacity	Cost
0	1	15	4
0	2	8	4
1	2	20	2
1	3	4	2
1	4	10	6
2	3	15	1
2	4	4	3
3	4	20	2
4	2	5	3

Table 3: Minimum cost flow test instance

An additional source s is linked to node 0 with capacity 20, and a sink t is connected from nodes 3 and 4.

The demand is:

$$20$$

The result obtained with Dijkstra and potentials is:

Arc	Flow	Capacity	Cost
$s \rightarrow 0$	20	20	0
$0 \rightarrow 1$	12	15	4
$0 \rightarrow 2$	8	8	4
$1 \rightarrow 2$	8	20	2
$1 \rightarrow 3$	4	4	2
$1 \rightarrow 4$	0	10	6
$2 \rightarrow 3$	12	15	1
$2 \rightarrow 4$	4	4	3
$3 \rightarrow 4$	11	20	2
$3 \rightarrow t$	5	5	0
$4 \rightarrow 2$	0	5	3
$4 \rightarrow t$	15	15	0
$t \rightarrow s$	0	1000	0

Table 4: Minimum cost flow result

The final cost is:

$$\begin{aligned}
 & 20 \times 0 + 12 \times 4 + 8 \times 4 + 8 \times 2 + 4 \times 2 + 0 \times 6 \\
 & + 12 \times 1 + 4 \times 3 + 11 \times 2 + 5 \times 0 + 0 \times 3 + 15 \times 0 \\
 & = 150
 \end{aligned}$$

So the minimum cost is:

150

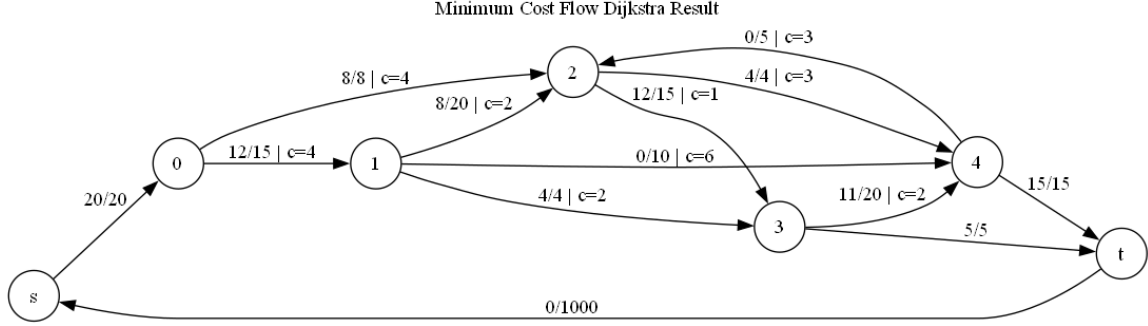


Figure 2: Graphviz visualization of the minimum cost flow result using Dijkstra with potentials

The Bellman-Ford version and the Dijkstra with potentials version produced the same total cost. This is a good validation of the implementation.

10 Negative Cycle Detection

A residual graph with a negative cycle means that the current flow is not optimal. Indeed, if a negative cycle exists, we can send flow around this cycle and decrease the total cost.

The negative cycle detection is implemented using Bellman-Ford.

The method is:

1. initialize all distances to zero,
2. relax all residual edges several times,
3. if an edge can still be relaxed after $|V| - 1$ iterations, then a negative cycle exists.

For the tested final minimum cost flow solutions, no negative cycle was found:

No negative cycle

This confirms the optimality of the obtained minimum cost flow.

11 Most Vital Arc

The most vital arc is the arc whose removal decreases the maximum flow the most.

The implemented method is simple:

1. compute the original maximum flow,
2. remove one arc,
3. recompute the maximum flow,

4. measure the reduction,
5. repeat for all arcs,
6. keep the arc with the largest reduction.

For the professor's maximum flow instance, the result is:

$$(0, 2)$$

The removal of this arc reduces the maximum flow by:

$$30$$

Therefore, the most vital arc is:

$$0 \rightarrow 2$$

Personal observation: this result makes sense because the arc $0 \rightarrow 2$ is saturated and has the largest outgoing capacity from the source.

12 Commands Used

The following commands were used to test the project:

```
python -m src.main maxflow examples/maxflow_example.json
python -m src.main mincost-bf examples/mincost_example.json
python -m src.main mincost-dijkstra examples/mincost_example.json

python -m src.main maxflow examples/teacher_maxflow.json
python -m src.main vital examples/teacher_maxflow.json

python -m src.main mincost-bf examples/teacher_mincost.json
python -m src.main mincost-dijkstra examples/teacher_mincost.json
```

Graphviz commands used to generate images:

```
dot -Tpng outputs/teacher_maxflow_maxflow_result.dot -o outputs/
teacher_maxflow.png
dot -Tpng outputs/teacher_mincost_mincost_dijkstra_result.dot -o
outputs/teacher_mincost_dijkstra.png
```

13 Difficulties Encountered

The main difficulty was not the maximum flow itself, but the management of the residual graph.

In particular:

- every edge must have a correct reverse edge,
- flow updates must update both forward and backward edges,

- minimum cost flow must use residual costs,
- Dijkstra cannot be used with negative costs without potentials.

Another difficulty was checking that both minimum cost flow versions return the same result. This helped me verify that the Bellman-Ford and Dijkstra implementations were coherent.

14 Conclusion

This project allowed me to implement several important algorithms from Operations Research.

The main lesson is that the residual graph is the central structure behind all flow algorithms. It is used to augment the flow, compute the minimum cut, update costs, and detect negative cycles.

The final implementation computes:

- maximum flow,
- minimum cut,
- minimum cost flow using Bellman-Ford,
- minimum cost flow using Dijkstra with potentials,
- negative cycle detection,
- most vital arc,
- Graphviz visualizations.

The tests given by the professor were successfully executed, and the visual results confirm the behavior of the algorithms.